From Prompt Engineering to Loop Engineering: A Five-Layer Taxonomy for Evaluating AI Systems Engineering Practice

Muhammad Faisal Laiq *Email: shmlaiq@gmail.com* *ORCID: 0009-0007-5779-9836*

Abstract — Large language model (LLM) application development has moved, in roughly five years, from hand-tuning a single input string to operating autonomous systems that plan, call tools, and revise their own output across multiple passes. Practitioners now use "prompt engineering," "context engineering," "agent engineering," and, more recently, "harness engineering" and "loop engineering" almost interchangeably, even though each term names a distinct engineering discipline with its own failure modes and its own way of measuring success. This paper proposes a five-layer taxonomy—Prompt Engineering, Context Engineering, Harness Engineering, Agent Engineering, and Loop Engineering—that separates these disciplines by what is actually being engineered at each stage: the input string, the surrounding information, the tool-and-permission scaffold, the delegation and planning behavior, and the iterative act-observe-verify cycle, respectively. For each layer we define boundary conditions relative to its neighbors, propose a concrete evaluation metric, and ground the layer in one published system from the literature and one first-person practitioner artifact drawn from an active multi-agent deployment. The paper is a taxonomy and case-study contribution, not a controlled experiment: its claims about the practitioner artifacts are illustrative, not measured, and we say so explicitly rather than let the distinction blur. We argue the taxonomy's principal practical use is diagnostic—identifying which layer a failing AI system is actually missing, particularly the common and under-diagnosed failure of building an agent without a loop, which produces confident but unverified output.

Index Terms — prompt engineering, context engineering, autonomous agents, multi-agent systems, software engineering taxonomy, large language models (LLMs)

I. Introduction

For most of computing history, giving a program intelligence meant writing the rules yourself: a system's behavior was exactly the logic an engineer had hand-coded into it, no more and no less, and improving that intelligence meant improving the code. Large language models broke that equivalence. A model's behavior comes from training, not from any rule a practitioner authored line by line, and the taxonomy this paper proposes exists because the disciplines that once governed hand-coded intelligence do not automatically transfer to a component nobody directly wrote.

Five years ago, getting useful behavior out of a language model meant finding the right words. Today it means operating a system: a model wrapped in retrieval, tools, delegation logic, and a verification loop that decides whether the system's own output is good enough to ship. The vocabulary practitioners use has not kept pace with that shift. A job posting for "prompt engineer" and a job posting for "AI agent engineer" frequently describe the same work, and a postmortem that concludes "the prompt needs work" often turns out, on inspection, to be a missing retrieval step, a tool that fails silently, or an agent that never checks its own answer before returning it.

This is not merely a terminology complaint. Teams that lack a shared vocabulary for *which* discipline is failing tend to apply the wrong fix—rewording a prompt when the real problem is that the model never sees the right context, or adding another layer of agent autonomy when the real problem is that no step in the pipeline ever verifies anything. Vendors selling "prompt engineering" courses, "context engineering" frameworks, and "agentic AI" platforms compound the confusion by using the terms as marketing labels rather than as engineering disciplines with distinct inputs, outputs, and failure signatures.

This paper's contributions are:

1) A five-layer taxonomy—Prompt Engineering, Context Engineering, Harness Engineering, Agent Engineering, Loop Engineering—that separates what is engineered at each stage of building an LLM-based system, with explicit boundary conditions between adjacent layers.
2) A proposed evaluation metric for each layer, designed to be diagnostic: given a failing system, the metrics are meant to help identify which layer actually needs work.
3) Ten case studies, two per layer—one drawn from the published literature and one a first-person practitioner artifact from an active multi-agent deployment—grounding each layer in something that actually exists rather than only in definition.
4) A discussion of the taxonomy's diagnostic use, including the specific and, we argue, systematically under-diagnosed failure of agent systems that lack any verification loop.

5) A mapping of each layer onto a pre-existing software engineering discipline—Design by Contract, configuration management, containerization, the actor model, and continuous delivery, respectively—that clarifies which difficulties are genuinely new to the LLM era and which are decades-old engineering concerns recurring under new vocabulary.

We are explicit about the shape of this contribution: this is a taxonomy and case-study paper, not a controlled experiment. We did not run a benchmark, recruit participants, or measure the proposed metrics across a sample of systems. Two of the five layers' practitioner case studies (Sections V-C through V-E) draw on artifacts the first author personally authored; we treat these as existence proofs that the taxonomy's categories are instantiable in real, currently-running practice, not as generalizable evidence that the taxonomy is optimal, and we return to this limitation directly in Section VII rather than leaving it implicit.

The remainder of this paper proceeds as follows. Section II reviews the literature underlying each of the five layers. Section III defines the taxonomy formally, including the boundary conditions between layers. Section IV proposes an evaluation metric per layer. Section V presents the ten case studies. Section VI discusses the taxonomy's practical, diagnostic use. Section VII states limitations and threats to validity. Section VIII traces each layer back to a pre-existing software engineering discipline that predates language models entirely. Section IX concludes with future work ordered by urgency.

II. Related Work

A. Prompt Engineering

The modern practice of prompt engineering traces to the observation that large pretrained language models can perform new tasks from a handful of in-context examples without any parameter update—the few-shot learning behavior demonstrated at scale by GPT-3 [1]. That single finding created an entire discipline: since the model itself is frozen, the only lever available to a practitioner is the string placed in front of it, and a substantial body of technique grew up around how to phrase, order, and exemplify that string. Chain-of-thought prompting [2] showed that asking a model to produce intermediate reasoning steps before its final answer measurably changes performance on multi-step reasoning tasks, and follow-up work showed the same effect could be triggered with a simple instruction rather than worked examples [3]. What unites this literature is that the model and its weights are treated as fixed; the entire engineering surface is the text of the prompt itself. None of it repairs a missing retrieval step. A rephrased prompt cannot supply information the model was never shown.

The prompt-engineering literature bifurcates into hand-crafted and automatically-searched technique. AutoPrompt [4] treats prompt discovery as a gradient-guided discrete search: it computes the gradient of task loss with respect to a one-hot embedding of each trigger-token position, shortlists candidate replacement tokens via a first-order Taylor approximation, and re-scores each candidate by substitution—a formal search procedure, but one that requires white-box gradient access and does not transfer to API-only models. The Automatic Prompt Engineer [5] instead treats instruction generation itself as a black-box optimization problem an LLM can solve for another LLM: a proposal model generates candidate instructions from a handful of demonstration pairs, each candidate is scored on held-out task performance, and the best candidates are iteratively refined by resampling variations. APE-discovered prompts outperformed human-written prompts on all 24 instruction-induction tasks tested and improved zero-shot chain-of-thought prompts specifically—direct evidence that the search space Section V-A's chain-of-thought case study explores by hand can itself be automated.

What counts as a "good prompt" is also not fixed across model variants of otherwise-identical weights. Instruction tuning [6] finetunes a base model on tens of task categories rewritten into multiple natural-language instruction templates, then evaluates zero-shot on entirely unseen categories; reinforcement learning from human feedback [7] goes further, training a reward model on human rankings of candidate outputs and optimizing the policy against that reward via PPO, a process that improved instruction-following, honesty, and toxicity simultaneously rather than only task-format compliance. Both processes change the frozen weights a prompt is written against, meaning a fixed prompt engineered for a base model is not guaranteed to transfer to an instruction-tuned or RLHF'd variant of the same underlying model—a second, independent way this layer's boundary is porous, distinct from its boundary with Layer 2.

Two findings ground the mechanistic and empirical basis of few-shot prompting itself. Induction heads [8]—a two-attention-head circuit that performs prefix-matching and copies the matched continuation into the output—form during a sharp phase transition in training that coincides with in-context learning ability improving sharply, the closest mechanistic, not merely behavioral, account of why a demonstration in the prompt changes model output at all. Earlier still, well-designed zero-shot prompts were shown to outperform few-shot prompts on the same task [9], suggesting few-shot examples often function to locate a capability already present from pretraining rather than to teach a new one—an early articulation of exactly this layer's definition, predating the term "prompt engineering" itself. A systematic survey [10] formalized "prompt-based learning" as a paradigm shift from pretrain-then-finetune, decomposing the design space by prompt shape (cloze vs. prefix) and by hand-crafted vs. automatically-searched vs. continuous or soft prompts—a finer-grained taxonomic effort within what this paper treats as a single layer.

Self-consistency [11] sits at this layer's upper boundary. Rather than a single greedy chain-of-thought decode, it samples a diverse set of reasoning paths at non-zero temperature and returns the answer with the most votes among them, reporting a 17.9-point gain on GSM8K over greedy decoding. By this paper's own Section III definition, majority-voting across a model's own samples is a self-consistency check, not an externally-verified one; it belongs conceptually closer to Layer 1's prompt-and-sample technique than to Layer 5's external-verifier requirement, and we flag that boundary ex-

plicitly here rather than let it blur unstated.

B. Context Engineering

Prompt engineering hits a hard ceiling the moment a task requires information the model was never trained on, or information that changes after training—a live database, a private document set, today's date. Retrieval-Augmented Generation [12] addressed this by pairing a frozen or fine-tuned generator with a retriever that pulls relevant passages into the model's input at inference time, moving the engineering problem from "what do I say" to "what does the model need to see." Anthropic's own practitioner guidance names this discipline directly, defining context engineering as curating and maintaining the optimal set of tokens available to the model at inference time and treating context as a limited resource to be spent deliberately rather than filled by default [13]. We use "context engineering" as the broader successor discipline to retrieval alone: the curation of everything that surrounds the instruction in the context window—conversation history, tool outputs, memory records, system instructions—and the decisions about what to include, what to omit, and how to order it so the model reasons over the right material. Context engineering treats the prompt's wording as secondary to the information assembled around it. That curation problem is not academic: models given long, retrieval-heavy contexts do not use every part of them equally, showing markedly worse recall for information placed in the middle of the context window than for information at either end [14], and a recent survey of production RAG systems catalogs the resulting engineering practice—chunking, re-ranking, and query rewriting—as responses to exactly that failure pattern [15]. Retrieval is not comprehension. A passage sitting in the context window does not mean the model actually used it.

The retrieval mechanism RAG depends on is itself an active engineering surface. Dense Passage Retrieval [16] replaced sparse lexical matching—BM25 and TF-IDF term overlap—with a dual-encoder architecture: separate BERT encoders embed the query and each candidate passage, and retrieval becomes nearest-neighbor search in the shared embedding space, reporting 9–19 absolute points of top-20 accuracy over BM25 and supplying the retriever the paper's already-cited RAG mechanism is built on top of. ColBERT [17] occupies a documented middle ground between DPR's fast-but-shallow bi-encoders and slow-but-accurate cross-encoders: it encodes query and passage into per-token vectors rather than one vector each, then scores relevance via a MaxSim operator that finds each query token's most similar passage token and sums those maxima, preserving token-level matching precision at roughly two orders of magnitude faster inference than cross-encoders. Retrieval quality is therefore a genuine speed/precision/index-size trade-off space, not a solved primitive this layer can treat as a black box.

Retrieval timing and self-assessment form a second axis of active engineering. Self-RAG [18] trains a model to emit special reflection tokens that decide, inline with normal generation, whether retrieval is needed for the current segment, whether a retrieved passage is relevant, and whether the generated text is actually supported by that passage—making retrieval adaptive rather than a single fixed pass, and making the resulting judgment inspectable rather than a hidden gate. Corrective RAG [19] instead inserts a lightweight evaluator between retrieval and generation that scores each retrieved document into a correct, ambiguous, or incorrect confidence band and triggers a different remedy per band—refining correct-but-noisy documents, falling back to web search for incorrect ones—a direct, citable answer to what a system should do once Section V-B's claim that "the retriever returned the wrong passage" is diagnosed. FLARE [20] takes a third position: rather than retrieving on a fixed schedule, it generates a tentative next sentence, checks that draft's token-level confidence, and only retrieves, using the low-confidence sentence itself as the query, when confidence falls below a threshold—making retrieval timing a continuous decision driven by the model's own uncertainty rather than a trained relevance classifier. We flag directly that Self-RAG's relevance and support checks are themselves self-assessment, not external verification, another instance of this taxonomy's recurring distinction surfacing inside the retrieval pipeline rather than only at the final output.

Context is also a resource to be spent deliberately, not merely retrieved. LLMLingua [21] scores each token's contribution to prompt perplexity using a small auxiliary model, on the premise that low-perplexity tokens carry little information the larger target model needs, then allocates a compression budget across the prompt's instruction, few-shot, retrieved-context, and question components and removes low-information tokens iteratively—reporting up to 20x compression with minimal task-performance loss, and operationalizing "what to include, what to omit" as an actual algorithm rather than only a design principle. MemGPT [22] borrows operating-system virtual memory paging directly: the model's context window is fast "main memory," an external store holds a much larger "disk" tier, and the model issues its own tool calls to page content in and out, with self-generated interrupts pausing generation mid-conversation to perform a memory operation. This is a genuine boundary case for the taxonomy: MemGPT's paging decisions are context-layer content selection, but they are executed through tool calls, a Layer 3 mechanism—an instance where the layers compose rather than cleanly separate, which we take as supporting rather than undermining Section VII's claim that layer boundaries are not always sharp.

Finally, a model's advertised context window and its effective usable context are measurably different quantities. Beyond the single-fact "needle in a haystack" test, RULER [23] adds multi-needle variants, multi-hop tracing across retrieved locations, and aggregation tasks requiring information combined from many places; evaluating 17 models advertising 32K-plus token windows, only about half maintain acceptable performance at the length they claim to support, with degradation sharpening at longer lengths and higher task complexity. This supplies the multi-model, multi-task empirical scope that this paper's earlier, narrower citation of the same phenomenon [14] does not by itself provide.

C. Harness Engineering

A model that can only read context and emit text cannot act on the world. Toolformer demonstrated that a language model could learn, largely on its own, when and how to invoke external tools (a calculator, a search API, a calendar) and incorporate their results into its output [24]. A broader survey of this line of work catalogs the surrounding design space—how a model decides which tool to call, how it formats the call, and how it incorporates the result—as "tool learning," a discipline distinct from the harness that executes the call safely once selected [25]. Tool-use and function-calling capability subsequently became a standard feature of production model APIs, and a substantial body of open-source and commercial "agent framework" tooling grew up around the scaffolding needed to execute a tool call safely—sandboxing, permissioning, retry logic, error surfacing back to the model. The Model Context Protocol formalized one version of this scaffolding as an open, client-server standard: rather than a bespoke integration per tool, a model-driven application connects through a single protocol to external data sources, tools, and predefined workflows exposed by independent servers [26]. Its adoption across multiple independent AI vendors and developer tools, not just its originating company, is itself evidence that the harness layer is a real, separately-engineered concern rather than an artifact of any one vendor's implementation. We adopt the term "harness engineering" for this scaffolding layer; unlike the other four layers, this term's academic footing is recent rather than established: Lin et al.'s 2026 study of automatic coding-agent harness evolution is, to our knowledge, the first paper to place "agentic harness engineering" in its own title, and industry practitioners had already converged on a compact one-line formulation of the same idea—"Agent = Model + Harness"—in the same period [27], [28]. We treat the term as an emerging one with academic usage only as recent as 2026, not as a decades-old settled category the way "prompt engineering" now is, and Section III's boundary conditions are this paper's own contribution to sharpening it rather than a restatement of prior consensus. A safe tool call is not a correct plan. Sandboxing a mistake still ships the mistake.

Concrete implementations of this scaffolding diverge on how tightly the action space is constrained. Gorilla [29] fine-tunes a model to generate accurate API calls against a retrieval-augmented index of API documentation, addressing two specific failure modes—imprecise argument generation and outright API hallucination—and updating what the model can call without retraining by updating the retrieved documentation instead. ToolLLM [30] constructs an instruction-tuning corpus of tool-use trajectories across more than 16,000 real-world APIs and introduces a depth-first-search decision tree that lets a model evaluate multiple candidate tool-selection paths rather than committing to one greedy chain, a direct answer to how a harness should structure search over a large tool registry. API-Bank [31] decomposes "can the model use a tool" into three separately-measured sub-capabilities—planning, retrieval, and invocation—and finds these do not move together across models (GPT-4 stronger at planning, GPT-3.5 at raw invocation), evidence that a harness's tool-registry concern is not a single monolithic capability but several distinct ones. At the more permissive end, CodeAct [32] replaces JSON-schema tool calls with executable Python code as the agent's unified action space, executed inside a Python interpreter—more expressive, since it supports dynamic composition and multi-turn state via variables, but it moves the harness's sandboxing and permission stakes considerably higher, reporting up to 20% higher task success across 17 models than JSON-style calling specifically because of that expressiveness. ToolkenGPT [33] answers a narrower registry-scaling question directly: representing each tool as a learned-embedding "toolken" token lets a frozen model invoke a new tool the same way it emits an ordinary vocabulary token, so the tool registry can grow without retraining the base model at all.

Orchestration is a distinct harness concern from execution. HuggingGPT [34] uses an LLM purely as a controller that decomposes a request into subtasks, selects a specialist model for each from a registry of model descriptions, sequences execution, and synthesizes results—keeping the harness's dispatch role structurally separate from generation. Chameleon [35] and RestGPT [36] solve an architecturally similar composition problem by different means: Chameleon's planner assembles plug-and-play tool modules via constraint inference over what each module produces and consumes, while RestGPT splits task decomposition from a dedicated executor module that handles REST call formatting and response parsing specifically—the same agent/harness split this paper's Section III draws, arrived at independently. That there are at least three non-identical architectural answers to essentially the same orchestration problem is itself evidence the harness layer is a real, unsettled design space rather than a solved primitive with one canonical implementation.

The permission and error-formatting half of this layer is not academic. Indirect prompt injection [37] demonstrates that malicious instructions embedded in content a tool-using model merely retrieves—not the user's direct prompt—can hijack the application: triggering unintended API calls, executing arbitrary code, and exfiltrating data, because the model cannot reliably distinguish data it is processing from instructions it should obey. The demonstration is not hypothetical; it was validated against real deployments including Bing Chat. This is the paper's clearest evidence that Layer 3's permission-system and error-formatting concerns exist because harnesses fail concretely when that boundary is porous, not merely because a taxonomy asserts the concern in the abstract.

D. Agent Engineering

Once a model has tools and a harness to execute them safely, a further question arises: which tool, in what order, and when should the process stop and ask a human? Anthropic's practitioner-oriented survey "Building Effective Agents" [38] draws a widely-cited distinction between *workflows*, in which the control flow is predefined by the engineer, and *agents*, in which the model itself directs its own process and tool use to accomplish a task. Open-source frameworks such as AutoGPT popularized the idea of a single model looping over

its own plan with minimal human oversight, while multi-agent frameworks including AutoGen [39] and CrewAI formalized patterns for multiple specialized agents collaborating, delegating subtasks, and communicating results. Generative Agents demonstrated a related but distinct point: giving a population of language-model-driven agents persistent memory, reflection, and planning was enough to produce believable emergent social behavior in a simulated environment, without any of the agents being told what to do turn by turn [40]. A broader survey organizes this literature around a common brain-perception-action framework spanning single-agent, multi-agent, and human-agent settings [41]. This literature is concerned with *decision-making behavior*—planning, task decomposition, delegation, escalation—layered on top of a harness that already exists. None of these frameworks verify their own output. Delegation is not correctness.

Concrete systems instantiate the workflow/agent distinction differently. Voyager [42] combines an automatic curriculum that proposes the next exploration goal to maximize learning progress with an expandable skill library that stores executable code for previously-learned behaviors, so capability accumulates and is reused rather than relearned each session—a code-level answer to how an agent's task-decomposition capability grows over a long-running deployment, directly relevant to this layer's persona-and-voice and delegation concerns. CAMEL [43] predates AutoGen's role-specialization pattern with "inception prompting": two agents are assigned complementary roles and converse autonomously toward task completion with no further human guidance, establishing role-specialized delegation's provenance earlier than the framework this paper already cites. Multiagent debate [44] takes the opposite structural approach—multiple instances of the identical model each propose an answer, observe each other's reasoning over several rounds, and revise their own position before a final convergent answer, an adversarial-collaborative delegation pattern rather than a role-specialized one, and a black-box wrapper compatible with any existing model.

Two frameworks build agent-to-agent verification directly into the delegation structure. MetaGPT [45] encodes standardized operating procedures mimicking a real software team—product manager, architect, engineer, QA—with each role explicitly verifying the previous role's output before passing work downstream, motivated directly by cascading hallucination across multi-agent handoffs. ChatDev [46] solves an architecturally distinct version of the same problem: a "chat chain" constrains what agents may communicate about at each development phase, paired with a "communicative dehallucination" technique that regulates inter-agent claims specifically to reduce fabrication passed downstream. Both sit exactly on this paper's Agent/Loop boundary—inter-role verification is a delegation-structure decision (Agent Engineering) that happens to also check individual outputs (Loop Engineering's concern)—and we treat their existence as evidence the boundary is a real design tension practitioners have already had to resolve, not an artifact of this paper's own categorization scheme.

Agent-layer capability is also measurably immature relative to prompt and context engineering. AgentBench [47] evaluates models across eight distinct interactive environments under realistic multi-step conditions and finds a large capability gap between top commercial and open-source models under 70B parameters, attributed specifically to poor long-term reasoning, decision-making, and instruction-following—an agent-layer deficit, not a raw language-modeling one. WebArena [48] supplies a concrete number for how large that gap can be in a realistic setting: GPT-4-based agents reach 14.41% task success on real, self-hosted websites against 78.24% human success on the same tasks. AgentVerse [49] studies group composition as a first-class variable rather than fixing it at the start of a task, and separately measures emergent social dynamics—both beneficial and counterproductive—among agents, work closer to what Section IV's delegation-correctness-versus-expert-decomposition metric would need to actually operationalize than a framework that treats group structure as a static design choice. Language Agent Tree Search [50] sharpens the Agent/Loop boundary further still, combining Monte Carlo Tree Search with LLM value functions and self-reflection to search over candidate action sequences using environmental feedback to guide which branches to expand—structurally close to Tree of Thoughts but applied to tool-using action selection rather than pure reasoning, blending planning search (an Agent-layer concern) with outcome-based pruning (a Loop-adjacent one) in a single system.

E. Loop Engineering

A system that plans and delegates still has no guarantee that any individual step's output is correct, and asking the model itself whether it is done is a weak check, since the same model that produced an error is often the one asked to certify it. ReAct interleaved reasoning traces with actions and their observed results, letting the model's next reasoning step be grounded in what actually happened rather than what it expected to happen [51]. Self-Refine showed that a model critiquing and rewriting its own output in successive passes, with no additional training and no external verifier at all, still produced measurable quality gains on a range of tasks—a useful contrast case for this layer, since it is closer to a loop's outer shape than to a loop's substance by our definition, precisely because the check being applied is the model's own judgment rather than anything external to it [52]. Reflexion extended this with an explicit self-critique step: on failure, the agent generates a verbal reflection on what went wrong and carries that reflection into its next attempt, functioning as a linguistic form of reinforcement learning without any weight update [53]. Tree of Thoughts generalized single-path reasoning into a search over multiple candidate reasoning paths, evaluated and pruned before a final answer is selected [54]. What these three lines of work share, and what we treat as the defining property of this layer, is a mechanical decide-to-continue step: the loop does not simply run once and stop, it observes a real result, checks it against something external to the model's own confidence, and only then decides whether to act again. A self-critique is not an external check. The same model that erred is still the one grading itself.

The central claim of this layer is no longer only a design position; it is a measured empirical result. Huang et al. [55]

report directly that large language models struggle to self-correct their own reasoning without external feedback, and that performance at times degrades after self-correction—not merely fails to improve, actively worsens. Kamoi et al.'s survey [56] confirms this is not an isolated finding: across the self-correction literature, no prior work demonstrates successful self-correction using only feedback a model generates about itself, outside narrow task types unusually suited to it, and many earlier "self-correction works" results rest on inflated or loose evaluation methodology. Together these two sources convert what this paper states as design intuition—a self-critique is not an external check—into the field's converging empirical consensus rather than one paper's assertion.

Verification mechanisms in the literature differ in exactly how external their check actually is. Self-consistency [11]—already discussed in Section II-A as a prompt-layer boundary case—samples multiple reasoning paths and votes on agreement, a weaker mechanism than an externally-grounded check since consensus among a model's own samples is still self-assessment, just aggregated rather than singular. Process reward models [57] occupy a genuine middle case: a separate model is trained on human step-level annotations to score each intermediate reasoning step rather than only the final answer, and process supervision significantly outperforms outcome-only supervision on the MATH benchmark—an external check, since the verifier is a distinct trained model, but a probabilistic rather than symbolic one. Constitutional AI [58] scopes the critique step against an external, written, fixed set of principles rather than the model's own free-floating judgment, generating self-critiques and revisions against that constitution in a supervised phase before training a preference model from AI-generated, not human-generated, judgments for an RL phase—a middle ground between the self-assessment Huang and Kamoi show fails and a fully external deterministic check. CRITIC [59] generalizes tool use explicitly into the verification role: the model generates an initial response, then invokes external tools specifically to validate or critique that response before revising—the tools function here as the external check itself, not merely as Section II-C's execution capability, a direct bridge between the harness and loop layers.

The cleanest and least ambiguous case is a fully symbolic, deterministic verifier. AlphaCode [60] generates a large pool of candidate solutions to a competitive-programming problem, runs each against test cases, and submits only survivors—pass/fail against a test suite admits no interpretation, and the system achieved an average top-54.3% ranking against fields of 5,000-plus Codeforces participants using exactly this filter. STaR [61] shows the same verifier-first principle applies to training as well as inference: an iterative bootstrap generates a rationale for each training question, keeps rationales that led to a correct final answer, regenerates, given the correct answer as a hint, for those that did not, and finetunes only on the rationales that survived that correctness filter—the same deterministic external check this paper's Section IV argues for at inference time, applied instead to what gets trained on.

An external verifier is also not automatically trustworthy simply because it is external. Scaling laws for reward-model overoptimization [62] show that even a trained proxy verifier can be gamed: optimizing against it via reinforcement learning or best-of-n sampling improves true reward only up to a point before degrading, a Goodhart's-Law failure mode, though larger reward models resist it more than smaller ones. The lineage from Christiano et al.'s original reward-modeling-from-preferences work [63] through InstructGPT's reward-model-plus-PPO pipeline [7] and WebGPT's citation-grounded, human-feedback-trained browsing agent [64] establishes that a verifier can itself be a trained model rather than only a symbolic rule—useful nuance for Section IV's evaluation framework, but one that inherits the same overoptimization caution rather than escaping it. AI Safety via Debate [65] supplies a structurally distinct verification pattern entirely: two agents argue opposing positions across alternating turns and a judge, human or in principle another model, decides which was more truthful, extending what a judge can verify from direct judgment to debate-assisted judgment under optimal play. We include it to note explicitly that "external check" need not mean a symbolic test; adversarial cross-examination counts too, provided the judge is not the same agent being checked, which is the one condition every mechanism surveyed in this subsection ultimately has to satisfy.

III. The Five-Layer Taxonomy

Each layer answers a different question about what is actually being engineered. The layers are cumulative in the sense that a system operating at Layer 5 still depends on Layers 1 through 4 functioning correctly underneath it—a well-verified loop cannot repair a harness that silently drops half its tool calls, any more than a beautifully worded prompt can repair a retrieval step that returns the wrong document. Fig. 1 depicts the stack.

Layer 1—Prompt Engineering. Unit of engineering: the input string. The practitioner treats the model and everything around it as fixed and asks only "how do I phrase this instruction, and which examples do I show, so that a single forward pass produces the output I want." *Boundary with Layer 2:* the moment the practitioner stops asking "how do I phrase this" and starts asking "what information does the model need that simply is not in the prompt yet," they have left Layer 1.

Layer 2—Context Engineering. Unit of engineering: the assembled context window—retrieved passages, tool outputs already collected, conversation history, memory records, system instructions—and the policy for what enters it, what is dropped, and in what order. *Boundary with Layer 3:* context engineering answers "what does the model see"; the moment the question becomes "what is the model allowed to *do*, and how is that action executed and checked," the practitioner has crossed into harness territory.

Layer 3—Harness Engineering. Unit of engineering: the scaffolding that determines what actions a model-driven process may take, how those actions are executed, sandboxed, retried on failure, and reported back to the model—independent of what task is being asked of the model at any given moment. Industry usage has converged on a compact one-line version of this same boundary: "Agent = Model + Harness" [28].

A permission system, a tool registry, a retry policy, and an error-formatting convention all live here—MCP servers are a concrete, standardized instance of exactly this registry-and-permission layer, not a competing concept. *Boundary with Layer 4:* harness engineering is infrastructure that exists whether or not anyone is currently using it; the moment engineering effort shifts to *which* tool the system should choose, in what order, and whether to delegate a subtask to another process, that is a decision-making concern, and belongs to the next layer up.

Layer 4—Agent Engineering. Unit of engineering: the decision-making behavior of an autonomous process built atop a harness—task decomposition, tool selection, delegation to sub-agents, persona and voice, and the explicit boundary of what the process may decide unsupervised versus what requires a human. *Boundary with Layer 5:* agent engineering specifies *what the system should decide to do*; it does not, by itself, guarantee that any individual decision or output was actually correct. The moment a mechanism is added whose job is to check a result against something external to the agent's own stated confidence—and to decide, based on that check, whether to try again—the system has entered loop-engineering territory.

Layer 5—Loop Engineering. Unit of engineering: the iterative act-observe-verify-decide cycle that governs whether an agent's own output gets checked, and how it gets revised, before it is accepted as final. The load-bearing property of this layer is a rule we adopt directly from ReAct- and Reflexion-style designs and from practitioner loop-engineering guidance we reviewed while preparing this paper: trust a deterministic, external check, never the agent's own report that it "looks done." A loop without such a check is, by our definition, not actually a loop—it is a chain that happens to run more than once.

No amount of engineering effort at a higher layer compensates for a defect at a lower one. A harness that silently swallows a tool's error message will produce confidently wrong agent decisions no matter how well the agent layer is designed, and a loop's verifier is only as good as the harness-level signal it has to check against.

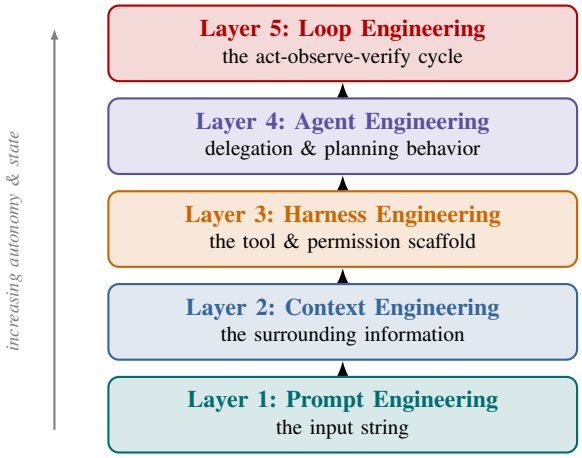


Fig. 1. The five-layer taxonomy, ordered by what is being engineered at each stage. Each layer subsumes the concerns of the layers below it; a defect at a lower layer cannot be repaired by better engineering at a higher one.

TABLE I
PER-LAYER DIAGNOSTIC METRICS.

Layer	Metric	Failure signature
Prompt	output variance across k equivalent rephrasings	correct on some phrasings, wrong on others, same information available
Context	retrieval precision/recall + context-window utilization	confident answer built on content never actually retrieved
Harness	tool-call success rate; error-recovery rate	a tool fails and the agent proceeds as if it succeeded
Agent	delegation correctness vs. expert decomposition	right tools exist, but work is routed to the wrong step
Loop	convergence rate; iterations-to-verified-success	success reported without the verifier ever running

IV. Evaluation Framework

A taxonomy is only useful in practice if it comes with a way to tell, from the outside, which layer is actually broken. We propose one diagnostic metric per layer, summarized in Table I and Fig. 2. Each is designed to isolate a failure at its own layer from a failure at a neighboring one, since misdiagnosis—treating a context problem as a prompt problem, or an agent problem as a loop problem—wastes engineering effort at the wrong point in the stack.

The loop-layer metric deserves its own emphasis, because it is where the taxonomy's central diagnostic claim lives: a loop is only as trustworthy as its verifier, and a verifier is only trustworthy if it is deterministic and external to the agent's own self-assessment. Per the layered-termination framing we adopt from loop-engineering practice, a robust loop needs more than one exit condition—a deterministic verifier for the primary pass/fail judgment, plus at least one safety exit such as a hard iteration cap, a budget guard on total tool calls or tokens spent, and no-progress detection that halts a loop whose recent passes changed nothing. A loop that has only a single exit—"stop when the model says it's done"—is not measuring convergence at all; it is measuring the model's own confidence, which is exactly the signal Reflexion-style designs treat as insufficient on its own.

Theoretical Analysis: Optimal Diagnostic Ordering. Section VI later argues for checking layers cheapest-first. That heuristic is a useful default. It is not the optimal one, and we derive the one that is here.

Complexity. With five layers, sequentially checking Layers 1 through 5 localizes any single-layer fault in at most $L = 5$ checks—an $O(L)$ worst case that holds regardless of which layer actually failed. An undifferentiated debugging process, one that does not decompose the system into independently checkable layers, has no comparable bound: the search space is whatever the engineer happens to think to try, and nothing in an unstructured process guarantees termination in a fixed number of steps. The taxonomy's five-layer decomposition is what converts an open-ended search into a bounded one.

Optimal ordering. Assume each layer i has a diagnostic-

Layer	Metric	Typical Failure Mode
Prompt	output variance across rephrasings	correct on some phrasings, wrong on others
Context	retrieval precision/recall; window utilization	answer built on content never retrieved
Harness	tool-call success & error-recovery rate	tool fails, agent proceeds as if it succeeded
Agent	delegation correctness vs. expert plan	tools work, but routed to the wrong step
Loop	convergence rate; iterations-to-verified-success	success reported without the verifier ever running

Fig. 2. Evaluation framework: one diagnostic metric and its typical failure mode per layer, color-coded consistently with Fig. 1.

check cost c_i (the engineer-time or compute needed to evaluate that layer's Table I metric) and a fault probability p_i—the probability the actual defect sits at layer i, with $\sum_i p_i = 1$ under a simplifying single-fault, independent-check assumption. Checking layers in ascending order of c_i/p_i—cost per unit of fault probability explained—minimizes the expected total diagnostic cost until the fault is found. A standard exchange argument proves this: take any ordering containing two adjacent checks, layer i immediately before layer j, where $c_i/p_i > c_j/p_j$. Swapping them so j runs first strictly reduces expected cost, because i's cost is paid in both orderings only in the case where neither i nor j has yet resolved the fault, while running the smaller-ratio check first resolves more expected fault probability per unit cost spent. Repeating this exchange on any non-optimal ordering strictly decreases expected cost at every step, so the ascending-ratio ordering is both a local and a global minimum.

Quantitative bound. Writing the layers in ascending-ratio order as $1, \ldots, 5$, the expected diagnostic cost is

$$\mathbb{E}[\text{cost}] = \sum_{i=1}^{5} c_i \left(1 - \sum_{j<i} p_j\right), \tag{1}$$

each layer's cost weighted by the probability the fault has not already been found by an earlier check. Equation (1) gives a concrete, checkable prediction: mechanically checking layers from cheapest-to-run to most-expensive-to-run, rather than by cost-to-probability ratio, pays strictly more in expectation whenever the cheapest layer to check is not also the layer most likely to be at fault—a gap this paper's own case studies suggest is common for Layer 5, where loop-layer checks are rarely the cheapest to run but catch the failure mode this paper argues is most often actually present and most often missed. Teams with even a rough prior on where their failures tend to cluster can use the ratio rule directly; teams without one default correctly to Section VI's cheapest-first heuristic, which is this rule's special case under a uniform fault-probability assumption.

V. Case Studies

For each layer we present two case studies: one drawn from a published system in the literature reviewed in Section II, and one a first-person practitioner artifact—a document currently governing how the first author's own AI tooling operates. The published case studies summarize prior findings at the level of detail appropriate to a taxonomy paper; readers who need exact reported figures should consult the cited primary sources directly. The practitioner case studies (C, D, and E below) are qualitative, illustrative descriptions of artifacts the first author personally authored and currently uses—not measured outcomes, not compared against alternative designs, and not evaluated by an independent party. We state this again here, in addition to Section I and Section VII, because it is the single most important caveat governing how these three subsections should be read.

A. *Prompt Engineering: Chain-of-Thought on Arithmetic Reasoning*

On the GSM8K grade-school arithmetic benchmark—originally introduced together with a trained verifier model as a way to check a solution's correctness independently of the model that produced it [66]—adding eight chain-of-thought exemplars to the prompt raised PaLM 540B's solve rate from an 18% standard-prompting baseline to 57% (58% when the model was additionally allowed a calculator), exceeding the prior fine-tuned-with-verifier state of the art of 55%—while the same manipulation produced a negligible or slightly negative effect on substantially smaller models in the same model family, only becoming a large, reliable gain in the roughly 62B-to-540B parameter range [2]. This is a clean Layer-1 case study. Nothing about the underlying model, its training data, or the problem's available context changed between conditions; the entire manipulation was the structure of the prompt, and the entire measured effect—an emergent-capability pattern rather than a uniform gain—is attributable to that structure.

B. *Context Engineering: Retrieval-Augmented Generation in Production*

Retrieval-augmented systems address knowledge-intensive tasks—open-domain question answering, fact verification, document-grounded dialogue—by retrieving passages from an external corpus at inference time and conditioning generation

on them, rather than relying solely on parametric knowledge frozen at training time [12]. A 2025 systematic review of production RAG deployments confirms that the operational engineering surface in practice is rarely the original generation model at all, but rather corpus construction, chunking strategy, embedding model choice, index freshness, and reranking—together with enterprise concerns such as security and scalability that a single research paper's evaluation setup does not have to address [67]. A system that performs poorly here typically does not fail because the prompt was worded badly; it fails because the retriever returned the wrong passage, or the right passage in a form too long or too fragmented for the model to use, which is a Layer-2 failure by our taxonomy regardless of how well-crafted the surrounding prompt is.

C. Harness Engineering: Agent Harnesses in the Literature and a Practitioner Artifact

Tool-use literature going back to Toolformer [24] established that a model can be given the *capability* to call external functions; production agent harnesses build the infrastructure around that capability—sandboxed execution, permission gating, structured error reporting back to the model, and retry policy—none of which changes what the model is being asked to accomplish.

As a first-person practitioner artifact, consider a containerization skill document used to constrain an AI coding assistant's Docker output. The skill defines a persona—"a DevOps engineer who optimizes container images for production deployments"—together with an explicit conflict-resolution priority order (security first, then runtime efficiency, then build convenience) and sixteen enumerated non-negotiable principles spanning build structure, package management, base-image selection, security hardening, runtime configuration, and external database connectivity. Before generating a Dockerfile, the skill runs seven analysis questions covering deployment target, base-image constraints, dependency management, large-file handling, security posture, health monitoring, and build context; it auto-detects Python version, framework, and package manager directly from repository files rather than asking the user; and it defines a resource-detection step that reads the host machine's available Docker memory and adapts container resource limits across four profiles—constrained, minimal, standard, and extended—before any configuration is generated. Delivery is gated by a six-item completion checklist (build succeeds; image size under 200 MB; health endpoint responds; container runs as a non-root user; no secrets appear in image history; the image runs on a different machine) that must be satisfied before the containerization task is considered done.

This is harness engineering under the Section III definition: none of the sixteen numbered requirements changes what the model is asked to produce—a working Dockerfile—but every one of them constrains how that output is checked, gated, and adapted to the runtime environment before it is accepted. It is worth naming a real limitation of this example rather than glossing over it: the constraint here is expressed as natural-language instruction text interpreted by the same model producing the output, not as an independently executing sandbox or permission system. A harness in the fuller sense—the sense that also governs, for instance, which files an assistant may edit or which shell commands require human approval—includes an enforcement layer that operates regardless of what the model decides. This skill document is closer to a *policy layer within* a harness than to the harness's full enforcement mechanism, and treating the two as interchangeable would overstate what the artifact actually demonstrates.

D. Agent Engineering: Multi-Agent Frameworks and an Operating Charter

Multi-agent frameworks such as AutoGen [39] formalize patterns for specialized agents to converse, delegate subtasks, and combine results, while Anthropic's practitioner guidance distinguishes agent designs, where the model directs its own process, from workflows, where control flow is predefined [38]. Both are concerned with decision-making behavior layered on top of an already-functioning harness.

As a first-person artifact at this layer, consider a persistent operating charter used to govern a coding assistant's delegation behavior across every session. The charter defines an identity for a top-level coordinating persona that reports only to a single named principal, and specifies that every other agent reports to that coordinator rather than to the principal directly. It authorizes the coordinator to construct new task-specific agents subject to the principal's approval, and specifies a sourcing procedure for skills the coordinator does not already have—locating a candidate skill from an external registry and gating its installation on two explicit checks, a popularity/trust signal and a full security audit, before use. It further specifies a workload-and-skill-fit rule governing how work is distributed once multiple agents exist. Layered on top of this identity specification is a mandatory six-step workflow—Plan, Review, Solution, Approve, Implement, Learn—with an explicit prohibition on skipping straight to implementation even for steps that look obvious, and a closing instruction to capture what was learned from any problem encountered during implementation so the same cost is not paid twice on a future task. Finally, the charter lists eight standing behavioral principles governing tone, response length, handling of ambiguity, and confidentiality, including an instruction that agents must not expose sensitive information shared in conversation to other agents without the principal's explicit instruction.

This document is agent engineering rather than harness engineering by our Section III boundary: it says nothing about what tools exist or how they are sandboxed, which is a harness concern handled elsewhere in the same tooling stack; it specifies how the coordinating agent should decide what to do, whom to delegate to, and when to stop and ask a human, which is exactly the planning-and-delegation behavior our taxonomy assigns to this layer. It also illustrates the agent/loop boundary drawn in Section III with unusual clarity: the six-step workflow constrains *sequencing and approval*—nothing proceeds from Plan to Implement without an explicit human Approve step—but it does not itself specify a deterministic verifier for any individual implementation step's correctness. That verification machinery is a separate artifact, discussed next, one layer up.

E. Loop Engineering: ReAct/Reflexion and a Fact-Checker Convergence Loop

ReAct grounds a model's next reasoning step in the observed result of its previous action rather than in the model's expectation of that result [51]; Reflexion adds an explicit self-critique step carried forward into the next attempt after a failure [53]; Tree of Thoughts generalizes this into a search over multiple candidate reasoning paths with evaluation and pruning before a final answer is chosen, nearly a twenty-fold improvement on the Game of 24 arithmetic puzzle over plain chain-of-thought prompting with the same underlying model (from 4% solved to 74%) [54]. All three treat "try again" as a decision to be earned by evidence, not a default.

The clearest first-person example of loop engineering in this practitioner's toolset is a document-production workflow that inserts a dedicated verification role between a drafting stage and a style-editing stage. After a draft section is produced, a separate Fact-Checker role reviews it and tags every issue it finds with one of three severities—Critical, Major, or Minor—and the workflow's exit condition is defined mechanically rather than left to the checker's overall impression: a section is marked clear for the next stage only once the Critical count and the Major count are both zero, while Minor issues are explicitly permitted to remain outstanding. Convergence is checked on every pass: the combined Critical-plus-Major count from the current iteration must be strictly lower than the count from the previous iteration, and if it is not, the workflow stops looping immediately—before any fixed cap is reached—and escalates the specific unresolved items to the human author rather than spending another identical-strength pass hoping for a different outcome. Independently of that convergence check, a hard ceiling of four iterations per section applies regardless of progress, and reaching that ceiling with an unresolved Critical or Major issue still outstanding is treated as a decision for the human to make, never something the workflow is permitted to wave through silently. Every iteration's findings report is preserved rather than overwritten, so the sequence of what was flagged and how it was resolved remains inspectable after the fact.

Mapped onto the vocabulary in Section IV, the deterministic verifier is the Critical=0-and-Major=0 rule, the no-progress-detection exit is the strictly-decreasing-count check, the budget guard is the four-iteration cap, and the human-escalation exit fires the moment the no-progress condition is met—all of the termination layers described in the loop-engineering literature we reviewed are present here in a document-production context rather than a benchmark-solving one. That is itself a modest finding worth stating plainly: the design vocabulary appears to transfer more readily across *task types*—arithmetic reasoning versus document editing—than it does across *published-versus-practitioner sources*, since everything described in this subsection is the workflow's own written specification, not an emergent property we independently measured.

Key Finding— The Fact-Checker loop's deterministic exit rule—Critical = 0 and Major = 0—is exactly the external, non-self-assessed check Section IV argues a genuine loop requires.

VI. Discussion

The taxonomy's main practical value is diagnostic. When a team reports "the agent isn't working," Section IV's per-layer metrics give an ordered checklist, cheapest to check first: is output unstable across equivalent phrasings (Layer 1), is the wrong information being retrieved or included (Layer 2), are tool calls failing silently (Layer 3), is work being routed to the wrong step (Layer 4), or is nothing ever actually verifying the result before it ships (Layer 5)? In our experience building the practitioner artifacts described in Section V, the last of these is the one most often skipped, and for a specific reason: a missing verifier does not look like an error. It looks like success. An agent with no loop layer will report that it finished, because nothing in its design distinguishes "I finished" from "I finished correctly."

This has a direct implication for tooling and hiring practice. Job postings, training curricula, and vendor platforms currently bundle all five disciplines under a single umbrella—"prompt engineering," or more recently "AI engineering" generally—in a way that obscures which specific skill a struggling team is actually missing. A team fluent in prompt and context engineering but with no experience designing a deterministic verifier will, by our taxonomy, systematically misattribute Layer-5 failures to Layers 1 or 2, because those are the layers they know how to inspect.

We want to state the central risk plainly rather than hedge it into vagueness: an agent layer without a loop layer is architecturally capable of taking any action its harness permits, but has no mechanism forcing it to check its own work before reporting success. This is not a hypothetical edge case—it is the default behavior of essentially every agent framework surveyed in Section II-D unless a verification step is deliberately added on top, and it is exactly the failure mode the loop-engineering literature's central rule—trust the deterministic check, never the agent's own "looks done" self-report—exists to close.

Key Finding— An agent layer without a loop layer can take any action its harness permits but has no mechanism forcing it to check its own work before reporting success.

VII. Limitations and Threats to Validity

This paper's practitioner case studies (Sections V-C, V-D, V-E) describe artifacts the first author personally authored and currently uses; they are illustrative single-instance descriptions, not measured outcomes, not compared against alternative designs, and not evaluated by any independent party. They demonstrate that the taxonomy's categories are instantiable in real, currently-operating practice—they do not demonstrate that the categories are optimal, complete, or generalize to other practitioners' systems.

The author proposing this taxonomy is also the author of the practitioner artifacts used to illustrate it. That is a framing-bias risk worth naming rather than disguising: a reader should

treat Sections V-C through V-E as existence proofs offered in good faith, not as endorsements of the taxonomy's correctness by an independent party.

Layer boundaries are not always sharp in real systems. A single engineering change—adding a persistent memory store to an agent, for instance—can simultaneously affect what we define as the context layer and the agent layer, and practitioners should treat the taxonomy as a descriptive lens for organizing engineering effort, not as a strict partition every real system cleanly respects.

The five evaluation metrics proposed in Section IV are conceptual instruments intended to guide diagnosis; this paper does not benchmark them against each other, validate them on a shared dataset, or report their reliability across raters. Doing so is explicitly future work, not a claim made here.

Finally, the published case studies in Section V summarize prior findings at a level of detail appropriate to a taxonomy paper's scope. Readers who need exact reported numbers, dataset splits, or statistical significance results should consult the cited primary sources directly rather than treat this paper's summaries as a substitute.

VIII. Traditional Programming Antecedents

None of the five concerns this taxonomy names are new to computing. Each has a direct antecedent in software engineering practice that predates language models by decades, and naming that antecedent does two things at once: it grounds the taxonomy in established discipline rather than presenting it as invented from nothing, and it clarifies what is actually novel about the LLM-era version—which, in every case below, turns out to be a change in *failure mode* rather than a change in the underlying engineering concern.

Layer 1 and Design by Contract. Meyer's Design by Contract specifies a function's preconditions and postconditions as an explicit, checkable interface: violate the contract, and the runtime halts with a reproducible, attributable error [68]. Prompt engineering asks a structurally similar question—what exact input reliably produces the exact output I want—but without an enforcement mechanism. A traditional contract violation is loud. A prompt that silently produces a different, equally fluent, wrong answer under a rephrasing is not; nothing in the system halts or flags it, which is precisely the failure mode Section IV's prompt-layer metric (output variance across rephrasings) exists to surface after the fact rather than prevent in advance.

Layer 2 and configuration management. The Twelve-Factor App methodology's third factor is to store configuration in the environment, separate from code, so that identical code behaves correctly across deployments [69]. Context engineering shares the underlying goal—keep the volatile, environment-specific material external and swappable—but not the failure guarantee. A twelve-factor application with missing configuration typically fails immediately and loudly: a connection refused, a null environment variable, a crash on startup. A language model given the wrong or insufficient context does not crash; it answers anyway, fluently, on the wrong material, which is why Section V-B's case study treats a wrong-passage retrieval failure as observationally indistinguishable from success until someone checks the source.

Layer 3 and containerization. A Docker container's isolation boundary is enforced by the operating system kernel—namespaces and control groups—independent of anything the contained process itself wants to do [70]. Harness engineering for language-model agents aspires to the same property, and Section V-C's practitioner artifact is worth revisiting here directly: its sixteen non-negotiable principles are enforced by the same model they constrain interpreting natural-language instruction text, not by an independent kernel-level mechanism. That is a materially weaker isolation boundary than a namespace, and it is the layer where this paper's traditional antecedent and its LLM-era descendant diverge the most.

Layer 4 and the actor model. Hewitt, Bishop, and Steiger's actor formalism defines computation as independent actors that communicate only through asynchronous messages, with each actor's internal state opaque to every other actor—a discipline that scales because actors never share mutable state [71]. Agent engineering borrows the vocabulary of delegation from this tradition but, in most current practice, not the isolation property: sub-agents in common multi-agent frameworks, and in the operating-charter artifact described in Section V-D, typically share a conversation history, a filesystem, or both. That is closer to shared-memory concurrency wearing actor-model terminology than to the actor model's actual message-passing discipline, and the distinction matters operationally—shared state between agents is exactly the condition under which one agent's error propagates silently into another's context.

Layer 5 and continuous delivery. A continuous delivery pipeline does not promote a build to the next environment until it clears a deterministic gate—a test suite, a linter, a security scan—and critically, the engineer who wrote the change does not get to unilaterally decide the gate is satisfied [72]. Of the five layers, this is the one whose LLM-era descendant matches its traditional antecedent most faithfully rather than least: the Fact-Checker convergence loop described in Section V-E is, structurally, a continuous delivery pipeline in which the artifact under test is a drafted document instead of a compiled binary, right down to the deterministic pass/fail gate, the prohibition on the same actor certifying its own output, and the escalation path when the gate cannot be satisfied automatically.

Read together, the five comparisons in this section support a claim narrower and more useful than "AI engineering is entirely new": the engineering concerns are old, the enforcement mechanisms traditional software engineering built for them were designed around deterministic components that fail loudly, and the specific difficulty of building reliable LLM-based systems is that the same concerns now sit on top of a component that tends to fail quietly instead. That reframing is itself the practical argument for Layer 5: a loop-engineering verifier is, in effect, an attempt to re-import the loud-failure guarantee that Design by Contract, twelve-factor configuration, kernel-level sandboxing, and the actor model each got for free from deterministic execution, and that a language model does not provide on its own.

Key Finding— Loop-engineering verifiers exist to re-import, for language models, the loud-failure guarantee that Design by Contract, twelve-factor configuration, kernel sandboxing, and the actor model each got for free from deterministic execution.

IX. Conclusion

We proposed a five-layer taxonomy—Prompt, Context, Harness, Agent, and Loop Engineering—separating what is actually engineered at each stage of building an LLM-based system.

Each layer gets a boundary condition relative to its neighbors, a proposed diagnostic metric, and grounding in ten case studies: five drawn from the published literature and five first-person practitioner artifacts. The taxonomy's principal use is diagnostic—helping a team locate which layer a failing system is actually missing, rather than defaulting to the layer they happen to already know how to fix. We paid particular attention to the risk of building agent systems that never verify their own output, since that failure is architecturally silent: it presents as success.

Future work is not symmetric, and treating it as three equally-weighted items would misrepresent how urgent each gap actually is. The five proposed metrics need empirical validation first—against a shared benchmark, or failing that, a corpus of real production incidents—because right now they are proposed instruments, not measured results, and that is this paper's most consequential open limitation. A formal cost model of engineering effort across layers matters next: our diagnostic argument in Section VI is currently qualitative, and a quantified trade-off between the cost of adding a verifier and the cost of an undetected Layer-5 failure would make the diagnostic use directly actionable rather than merely suggestive. Whether additional layers emerge above Loop Engineering, as multi-agent systems begin verifying each other's loops instead of relying on one deterministic check, is a real question worth asking. It is also the one this paper is least equipped to answer, since the five-layer stack proposed here does not yet have the vocabulary for it.

Acknowledgments

No external funding was received for this work. In accordance with IEEE's generative AI disclosure policy, the author discloses that an AI-LLM was used to assist with literature search and citation verification, drafting and structuring the manuscript text, and fact-checking and editorial review during the preparation of this paper. All AI-assisted output was reviewed, verified against primary sources, and edited by the author, who takes full responsibility for the accuracy and content of the final manuscript. No AI system is credited as an author.

References

[1] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877–1901, 2020.

[2] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 24824–24837, 2022.

[3] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 22199–22213, 2022.

[4] T. Shin, Y. Razeghi, R. L. Logan IV, E. Wallace, and S. Singh, "Autoprompt: Eliciting knowledge from language models with automatically generated prompts," *arXiv preprint arXiv:2010.15980*, 2020.

[5] Y. Zhou, A. I. Muresanu, Z. Han, K. Paster, S. Pitis, H. Chan, and J. Ba, "Large language models are human-level prompt engineers," *arXiv preprint arXiv:2211.01910*, 2022.

[6] J. Wei, M. Bosma, V. Y. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai, and Q. V. Le, "Finetuned language models are zero-shot learners," *arXiv preprint arXiv:2109.01652*, 2021.

[7] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe, "Training language models to follow instructions with human feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.

[8] C. Olsson, N. Elhage, N. Nanda, N. Joseph, N. DasSarma, T. Henighan, B. Mann, A. Askell, Y. Bai, A. Chen, T. Conerly, D. Drain, D. Ganguli, Z. Hatfield-Dodds, D. Hernandez, S. Johnston, A. Jones, J. Kernion, L. Lovitt, K. Ndousse, D. Amodei, T. Brown, J. Clark, J. Kaplan, S. McCandlish, and C. Olah, "In-context learning and induction heads," *arXiv preprint arXiv:2209.11895*, 2022.

[9] L. Reynolds and K. McDonell, "Prompt programming for large language models: Beyond the few-shot paradigm," in *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*, 2021.

[10] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, "Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing," *arXiv preprint arXiv:2107.13586*, 2021.

[11] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," *arXiv preprint arXiv:2203.11171*, 2022.

[12] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.

[13] P. Rajasekaran, E. Dixon, C. Ryan, and J. Hadfield, "Effective context engineering for AI agents." Anthropic Engineering Blog, 2025. Published September 29, 2025. Available: https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents.

[14] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *arXiv preprint arXiv:2307.03172*, 2023.

[15] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2023.

[16] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 6769–6781, 2020.

[17] O. Khattab and M. Zaharia, "ColBERT: Efficient and effective passage search via contextualized late interaction over BERT," in *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2020.

[18] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," *arXiv preprint arXiv:2310.11511*, 2023.

[19] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, "Corrective retrieval augmented generation," *arXiv preprint arXiv:2401.15884*, 2024.

[20] Z. Jiang, F. F. Xu, L. Gao, Z. Sun, Q. Liu, J. Dwivedi-Yu, Y. Yang, J. Callan, and G. Neubig, "Active retrieval augmented generation," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[21] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, "LLMLingua: Compressing prompts for accelerated inference of large language models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[22] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," *arXiv preprint arXiv:2310.08560*, 2023.

[23] C.-P. Hsieh, S. Sun, S. Kriman, S. Acharya, D. Rekesh, F. Jia, Y. Zhang, and B. Ginsburg, "RULER: What's the real context size of your long-context language models?," *arXiv preprint arXiv:2404.06654*, 2024.

[24] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[25] Y. Qin, S. Hu, Y. Lin, W. Chen, N. Ding, G. Cui, Z. Zeng, Y. Huang, C. Xiao, C. Han, Y. R. Fung, Y. Su, H. Wang, C. Qian, R. Tian, K. Zhu, S. Liang, X. Shen, B. Xu, Z. Zhang, Y. Ye, B. Li, Z. Tang, J. Yi, Y. Zhu, Z. Dai, L. Yan, X. Cong, Y. Lu, W. Zhao, Y. Huang, J. Yan, X. Han, X. Sun, D. Li, J. Phang, C. Yang, T. Wu, H. Ji, Z. Liu, and M. Sun, "Tool learning with foundation models," *arXiv preprint arXiv:2304.08354*, 2023.

[26] Anthropic, "Introducing the model context protocol." Anthropic News, 2024. Published November 25, 2024. Available: https://www.anthropic.com/news/model-context-protocol.

[27] J. Lin, S. Liu, C. Pan, L. Lin, S. Dou, Z. Xi, X. Huang, H. Yan, Z. Han, T. Gui, and Y.-G. Jiang, "Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses," *arXiv preprint arXiv:2604.25850*, 2026.

[28] V. Trivedy, "The anatomy of an agent harness." LangChain Blog, 2026. Published March 10, 2026. Available: https://www.langchain.com/blog/the-anatomy-of-an-agent-harness.

[29] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, "Gorilla: Large language model connected with massive APIs," *arXiv preprint arXiv:2305.15334*, 2023.

[30] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, L. Hong, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun, "ToolLLM: Facilitating large language models to master 16000+ real-world APIs," *arXiv preprint arXiv:2307.16789*, 2023.

[31] M. Li, Y. Zhao, B. Yu, F. Song, H. Li, H. Yu, Z. Li, F. Huang, and Y. Li, "API-Bank: A comprehensive benchmark for tool-augmented LLMs," *arXiv preprint arXiv:2304.08244*, 2023.

[32] X. Wang, Y. Chen, L. Yuan, Y. Zhang, Y. Li, H. Peng, and H. Ji, "Executable code actions elicit better LLM agents," *arXiv preprint arXiv:2402.01030*, 2024.

[33] S. Hao, T. Liu, Z. Wang, and Z. Hu, "ToolkenGPT: Augmenting frozen language models with massive tools via tool embeddings," *arXiv preprint arXiv:2305.11554*, 2023.

[34] Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang, "HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face," *arXiv preprint arXiv:2303.17580*, 2023.

[35] P. Lu, B. Peng, H. Cheng, M. Galley, K.-W. Chang, Y. N. Wu, S.-C. Zhu, and J. Gao, "Chameleon: Plug-and-play compositional reasoning with large language models," *arXiv preprint arXiv:2304.09842*, 2023.

[36] Y. Song, W. Xiong, D. Zhu, W. Wu, H. Qian, M. Song, H. Huang, C. Li, K. Wang, R. Yao, Y. Tian, and S. Li, "RestGPT: Connecting large language models with real-world RESTful APIs," *arXiv preprint arXiv:2306.06624*, 2023.

[37] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," *arXiv preprint arXiv:2302.12173*, 2023.

[38] Erik S. and B. Zhang, "Building effective agents." Anthropic Engineering Blog, 2024. Published December 19, 2024. Available: https://www.anthropic.com/research/building-effective-agents.

[39] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "Autogen: Enabling next-gen LLM applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023.

[40] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," *arXiv preprint arXiv:2304.03442*, 2023.

[41] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou, R. Zheng, X. Fan, X. Wang, L. Xiong, Y. Zhou, W. Wang, C. Jiang, Y. Zou, X. Liu, Z. Yin, S. Dou, R. Weng, W. Cheng, Q. Zhang, W. Qin, Y. Zheng, X. Qiu, X. Huang, and T. Gui, "The rise and potential of large language model based agents: A survey," *arXiv preprint arXiv:2309.07864*, 2023.

[42] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," *arXiv preprint arXiv:2305.16291*, 2023.

[43] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, "CAMEL: Communicative agents for "mind" exploration of large language model society," *arXiv preprint arXiv:2303.17760*, 2023.

[44] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, "Improving factuality and reasoning in language models through multiagent debate," *arXiv preprint arXiv:2305.14325*, 2023.

[45] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "MetaGPT: Meta programming for a multi-agent collaborative framework," *arXiv preprint arXiv:2308.00352*, 2023.

[46] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, "ChatDev: Communicative agents for software development," *arXiv preprint arXiv:2307.07924*, 2023.

[47] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang, "AgentBench: Evaluating LLMs as agents," *arXiv preprint arXiv:2308.03688*, 2023.

[48] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig, "WebArena: A realistic web environment for building autonomous agents," *arXiv preprint arXiv:2307.13854*, 2023.

[49] W. Chen, Y. Su, J. Zuo, C. Yang, C. Yuan, C.-M. Chan, H. Yu, Y. Lu, Y.-H. Hung, C. Qian, Y. Qin, X. Cong, R. Xie, Z. Liu, M. Sun, and J. Zhou, "AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors," *arXiv preprint arXiv:2308.10848*, 2023.

[50] A. Zhou, K. Yan, M. Shlapentokh-Rothman, H. Wang, and Y.-X. Wang, "Language agent tree search unifies reasoning, acting, and planning in language models," *arXiv preprint arXiv:2310.04406*, 2023.

[51] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023. arXiv preprint arXiv:2210.03629, 2022.

[52] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, "Self-refine: Iterative refinement with self-feedback," *arXiv preprint arXiv:2303.17651*, 2023.

[53] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[54] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[55] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou, "Large language models cannot self-correct reasoning yet," *arXiv preprint arXiv:2310.01798*, 2023.

[56] R. Kamoi, Y. Zhang, N. Zhang, J. Han, and R. Zhang, "When can LLMs actually correct their own mistakes? a critical survey of self-correction of LLMs," *arXiv preprint arXiv:2406.01297*, 2024.

[57] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe, "Let's verify step by step," *arXiv preprint arXiv:2305.20050*, 2023.

[58] Y. Bai, S. Kadavath, S. Kundu, A. Askell, J. Kernion, A. Jones, A. Chen, A. Goldie, A. Mirhoseini, C. McKinnon, C. Chen, C. Olsson, C. Olah, D. Hernandez, D. Drain, D. Ganguli, D. Li, E. Tran-Johnson, E. Perez, J. Kerr, J. Mueller, J. Ladish, J. Landau, K. Ndousse, K. Lukosuite, L. Lovitt, M. Sellitto, N. Elhage, N. Schiefer, N. Mercado, N. DasSarma, R. Lasenby, R. Larson, S. Ringer, S. Johnston, S. Kravec, S. El Showk, S. Fort, T. Lanham, T. Telleen-Lawton, T. Conerly, T. Henighan, T. Hume, S. R. Bowman, Z. Hatfield-Dodds, B. Mann, D. Amodei, N. Joseph, S. McCandlish, T. Brown, and J. Kaplan, "Constitutional AI: Harmlessness from AI feedback," *arXiv preprint arXiv:2212.08073*, 2022.

[59] Z. Gou, Z. Shao, Y. Gong, Y. Shen, Y. Yang, N. Duan, and W. Chen, "CRITIC: Large language models can self-correct with tool-interactive critiquing," *arXiv preprint arXiv:2305.11738*, 2023.

[60] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. Dal Lago, T. Hubert, P. Choy, C. de Masson d'Autume, I. Babuschkin, X. Chen, P.-S. Huang, J. Welbl, S. Gowal, A. Cherepanov, J. Molloy, D. J. Mankowitz, E. S. Robson, P. Kohli, N. de Freitas, K. Kavukcuoglu, and O. Vinyals, "Competition-level code generation with AlphaCode," *arXiv preprint arXiv:2203.07814*, 2022.

[61] E. Zelikman, Y. Wu, J. Mu, and N. D. Goodman, "STaR: Bootstrapping reasoning with reasoning," *arXiv preprint arXiv:2203.14465*, 2022.
[62] L. Gao, J. Schulman, and J. Hilton, "Scaling laws for reward model overoptimization," *arXiv preprint arXiv:2210.10760*, 2022.
[63] P. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei, "Deep reinforcement learning from human preferences," *arXiv preprint arXiv:1706.03741*, 2017.
[64] R. Nakano, J. Hilton, S. Balaji, J. Wu, L. Ouyang, C. Kim, C. Hesse, S. Jain, V. Kosaraju, W. Saunders, X. Jiang, K. Cobbe, T. Eloundou, G. Krueger, K. Button, M. Knight, B. Chess, and J. Schulman, "WebGPT: Browser-assisted question-answering with human feedback," *arXiv preprint arXiv:2112.09332*, 2021.
[65] G. Irving, P. Christiano, and D. Amodei, "AI safety via debate," *arXiv preprint arXiv:1805.00899*, 2018.
[66] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, "Training verifiers to solve math word problems," *arXiv preprint arXiv:2110.14168*, 2021.
[67] A. J. Oche, A. G. Folashade, T. Ghosal, and A. Biswas, "A systematic review of key retrieval-augmented generation (RAG) systems: Progress, gaps, and future directions," *arXiv preprint arXiv:2507.18910*, 2025.
[68] B. Meyer, "Applying 'design by contract'," *Computer*, vol. 25, no. 10, pp. 40–51, 1992.
[69] A. Wiggins, "The twelve-factor app." Heroku, 2017. Available: https://12factor.net/.
[70] D. Merkel, "Docker: Lightweight linux containers for consistent development and deployment," *Linux Journal*, no. 239, 2014.
[71] C. Hewitt, P. Bishop, and R. Steiger, "A universal modular ACTOR formalism for artificial intelligence," in *Proceedings of the 3rd International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 235–245, 1973.
[72] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases Through Build, Test, and Deployment Automation*. Addison-Wesley, 2010.



Muhammad Faisal Laiq received the master's degree in telecommunication.

He has served in the government sector since 2006 and is currently Additional Director ICT at a public-sector university and hospital in Pakistan, where he leads digital transformation and ICT infrastructure. He is also an agentic AI engineer and independent researcher. His AI work centers on AI-driven development (AIDD), loop engineering, and hierarchical multi-agent systems ("agentic fleets") built with Python, FastAPI, Docker, and Kubernetes, in which a chief agent proposes specialized sub-agents but must obtain human approval before creating or activating them. His research includes the paper *Artificial Intelligence in Future Warfare: Ethical Frameworks and the Regulation of Lethal Autonomous Weapons*, and covers AI governance, forward-deployed engineering in the AI industry, biocomputing as an alternative to silicon-based AI, the environmental cost of AI resource consumption, and operating systems for cloud-accessible quantum computers.